

7장 트랜스포머

트랜스포머 (Transformer)

2017년 코넬 대학의 아시시 바스와니 등의 연구 그룹이 발표한 논문을 통해 소개된 신경망 아키텍처

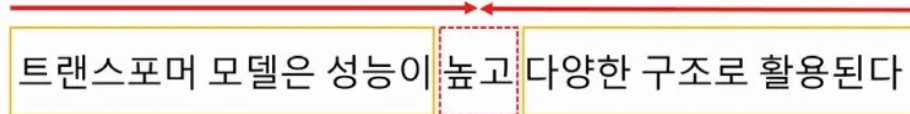
- 병렬로 입력 시퀀스 처리
- 긴 시퀀스/대용량 데이터셋 의 경우 유리
- **셀프 어텐션 (Self-Attention) 기반** : 순차 처리나 반복 연결에 의존하지 않고 입력 토큰 간의 관계를 직접 처리하고 이해할 수 있도록 함
- 언어 모델링 및 텍스트 분류와 같은 작업에서 매우 효과적, 광범위한 자연어 처리 작업에서 높은 효율. (ex. 기계 번역, 언어 모델링, 텍스트 요약 등)
- 오토 인코딩과 자기 회귀



오토 인코딩 (Auto Encoding)

- 랜덤하게 문장의 일부를 빈칸 토큰으로 만들고 해당 빈칸에 어떤 단어가 적절한지 예측하는 작업 수행.
- 예측되는 토큰의 양옆에 있는 토큰들을 참조하기 때문에 양방향 구조를 가지면 이를 인코더(**Ecoder**) 라고 한다.

A) 양방향 구조



입력 : '트랜스포머 모델은 성능이', '다양한 구조로 활용된다'

출력 : '높고'



자기 회귀 (Auto Regressive)

- 이전 단어들이 주어졌을 때 다음 단어가 무엇인지 맞추는 작업 수행
- 예측되는 토큰의 왼쪽에 있는 토큰들만 참조하기 때문에 단방향 구조를 가지는 **디코더(Decoder)** 를 사용

B) 단방향 구조

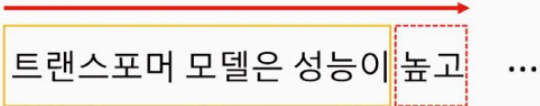


그림 7.1 트랜스포머 구조



입력 : '트랜스포머 모델은 성능이'

출력 : '높고'

Transformer

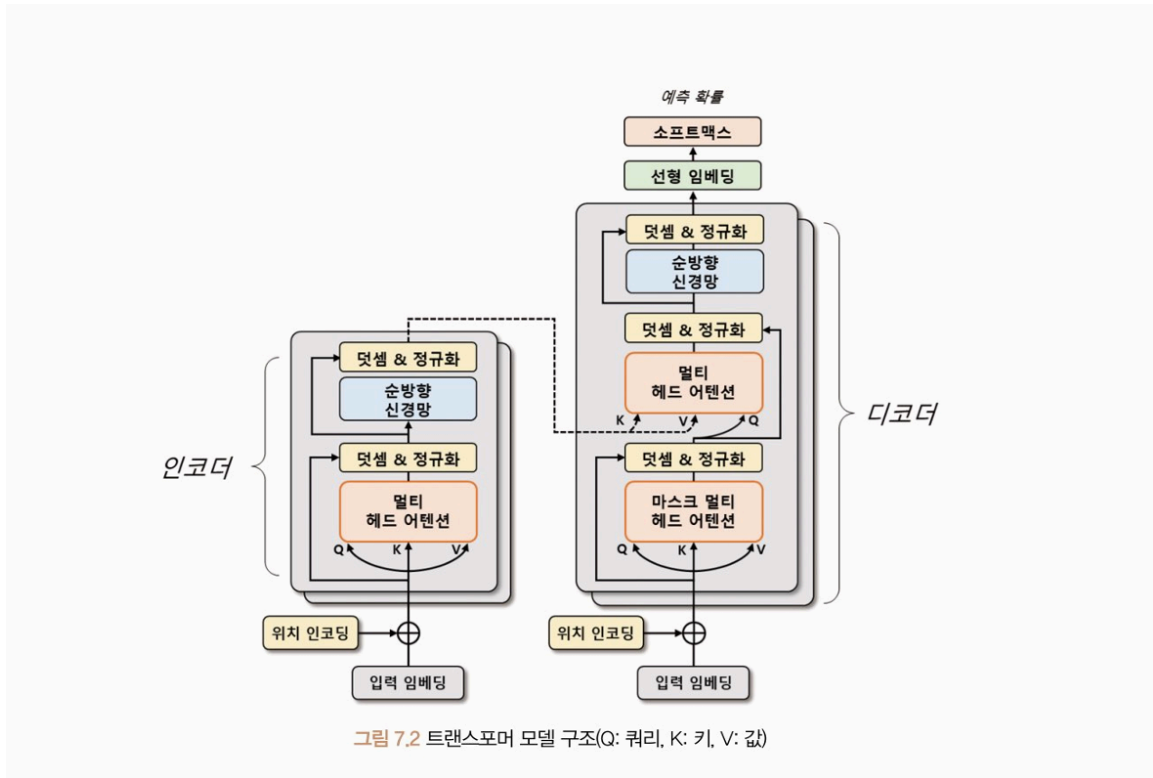
딥러닝 모델 중 하나로, 다양한 자연어 처리 분야에서 많은 성과를 내는 모델. 순환 신경망이나 합성곱 신경망과 달리 **어텐션 매커니즘(Attention Mechanism)** 만을 사용하여 시퀀스 임베딩 표현

- 어텐션 매커니즘

인코더와 디코더 단어 사이의 상관관계를 계산하여 중요한 정보에 집중.

→ 입력 시퀀스의 각 단어가 출력 시퀀스의 어떤 단어와 관련이 있는지를 파악한다.

- 트랜스포머 모델 구조



- 인코더, 디코더 두 부분 각각 N 개의 트랜스포머 블록으로 구성
- 트랜스포머 블록은 **멀티 헤드 어텐션**과 **순방향 신경망**으로 구성
 - 멀티 헤드 어텐션 (Multi Head Attention) : 입력 시퀀스에서 쿼리 (Query), 키(Key), 값(Value) 벡터를 정의해 입력 시퀀스들의 관계를 셀프 어텐션(Self Attention) 하는 벡터 표현 방법.
 - 이 과정에서 쿼리와 각 키의 유사도를 계산하고 해당 유사도를 가중치로 사용하여 값 벡터 합산, 어텐션 행렬 생성
 - 어텐션 행렬은 입력 시퀀스 각 단어의 임베딩 벡터를 대체
 - 순방향 신경망 : 이 과정에서 산출된 임베딩 벡터를 더욱 고도화하기 위해 사용. 여러 개의 선형 계층으로 이루어져 있으며, 입력 벡터에 가중치를 곱하고, 편향을 더하며, 활성화 함수를 적용하는 작업을 반복.
 - 이 과정에서 학습된 가중치들은 입력 시퀀스의 각 단어의 의미를 잘 파악할 수 있는 방식으로 갱신

입력 임베딩과 위치 인코딩

- 트랜스포머 모델에서 입력 시퀀스의 각 단어는 임베딩 처리되어 벡터 형태로 변환
- 트랜스포머 모델은 입력 시퀀스를 병렬 구조로 처리하므로 단어의 순서 정보 제공 X

→ 위치 정보를 임베딩 벡터에 추가해 순서 정보를 반영하는 작업 필요

→ **위치 인코딩** : 입력 시퀀스의 **순서 정보**를 모델에 전달하는 방법으로, 각 단어의 **위치 정보를 나타내는 벡터 위치 인코딩 벡터**를 더하여 임베딩 벡터에 위치 정보 반영

▶▶ 각 토큰의 위치를 각도로 표현해 sin 함수와 cos 함수로 위치 임베딩 벡터 계산

#위치 인코딩

```
import math
import torch
from torch import nn
from matplotlib import pyplot as plt

class PositionalEncoding(nn.Module): #입력 : 임베딩 차원(d_model), 최대 시퀀스 길이(max_len)
    def __init__(self, d_model, max_len, dropout=0.1):
        super().__init__()
        self.dropout = nn.Dropout(p=dropout)

        position = torch.arange(max_len).unsqueeze(1)
        div_term = torch.exp(
            torch.arange(0, d_model, 2) * (-math.log(10000.0) / d_model)
        )

        pe = torch.zeros(max_len, 1, d_model)
        pe[:, 0, 0::2] = torch.sin(position * div_term)
        pe[:, 0, 1::2] = torch.cos(position * div_term)
        self.register_buffer("pe", pe)

    def forward(self, x):
        x = x + self.pe[:x.size(0)]
        return self.dropout(x)

# 인코딩 시각화
encoding = PositionalEncoding(d_model=128, max_len=50)
plt.pcolormesh(encoding.pe.detach().numpy().squeeze(), cmap="RdBu")
plt.xlabel("Embedding Dimension")
plt.xlim((0, 128))
plt.ylabel("Position")
```

```
plt.colorbar()
plt.show()
```

- `PositionalEncoding` 클래스 : 입력으로 임베딩 차원 (`d_model`), 최대 시퀀스 (`max_len`) 받음

→ 입력 시퀀스의 위치마다 sin 과 cos 함수로 위치 인코딩 계산

$$PE(pos, 2i) = \sin(pos/10000^{2i/d_{model}})$$
$$PE(pos, 2i + 1) = \cos(pos/10000^{2i/d_{model}})$$

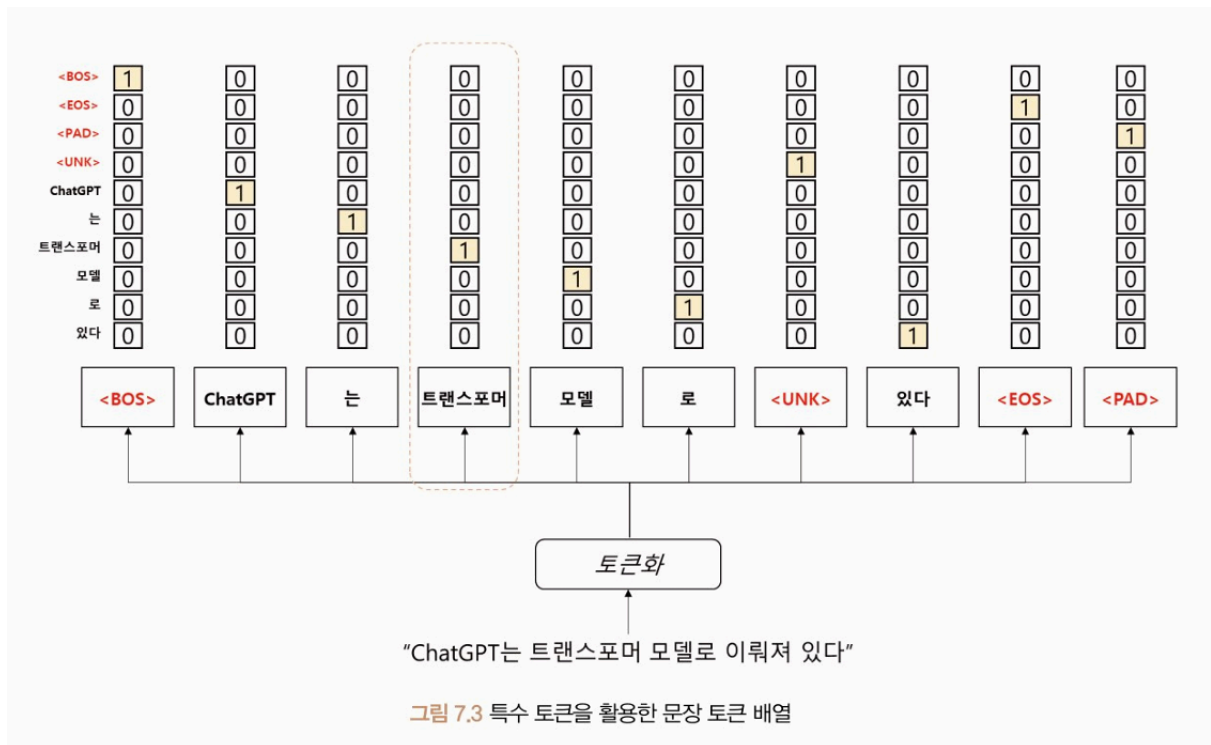
- `pos` : 입력 시퀀스에서 해당 단어의 위치를 나타냄
- `i` : 임베딩 벡터의 차원 인덱스를 의미

차원의 인덱스가 짝수라면 첫 번째 sin 함수 수식을 적용, 홀수라면 두 번째 cos 함수 수식 적용. $1/10000^{2i/d}$ 는 각도 정보로 변환하기 위한 스케일링 인자로, 각 위치에 대한 주기적인 신호 생성

- `PE` 변수의 텐서 차원은 [50, 1, 128] 이며 [최대 시퀀스, 1, 입력 임베딩 차원] 의미
- `register_buffer` 메서드 : 모델이 매개변수를 갱신하지 않도록 설정

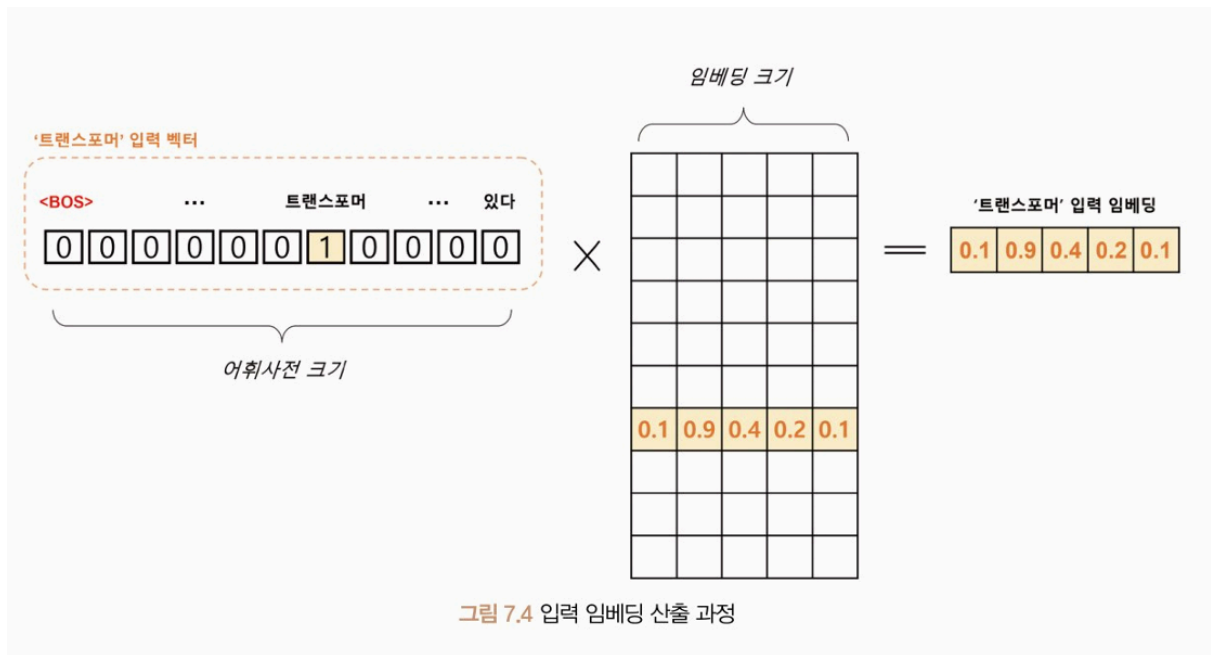
특수 토큰

- 트랜스포머는 단어 토큰 이외의 특수 토큰을 활용해 문장을 표현
- 입력 시퀀스의 시작과 끝을 나타내거나 **마스킹 영역**으로 사용



- **BOS(Beginning of Sentence)** : 문장의 시작
- **EOS(End of Sentence)** : 문장의 끝
- **UNK(Unknown)** : 어휘 사전에 없는 단어로, 모르는 단어를 의미
- **PAD (Padding)** : 모든 문장을 일정한 길이로 맞추기 위해 사용

- 문장 토큰 배열을 어휘 사전에 등장하는 위치에 원-핫 인코딩으로 표현 → 입력 임베딩으로 변환

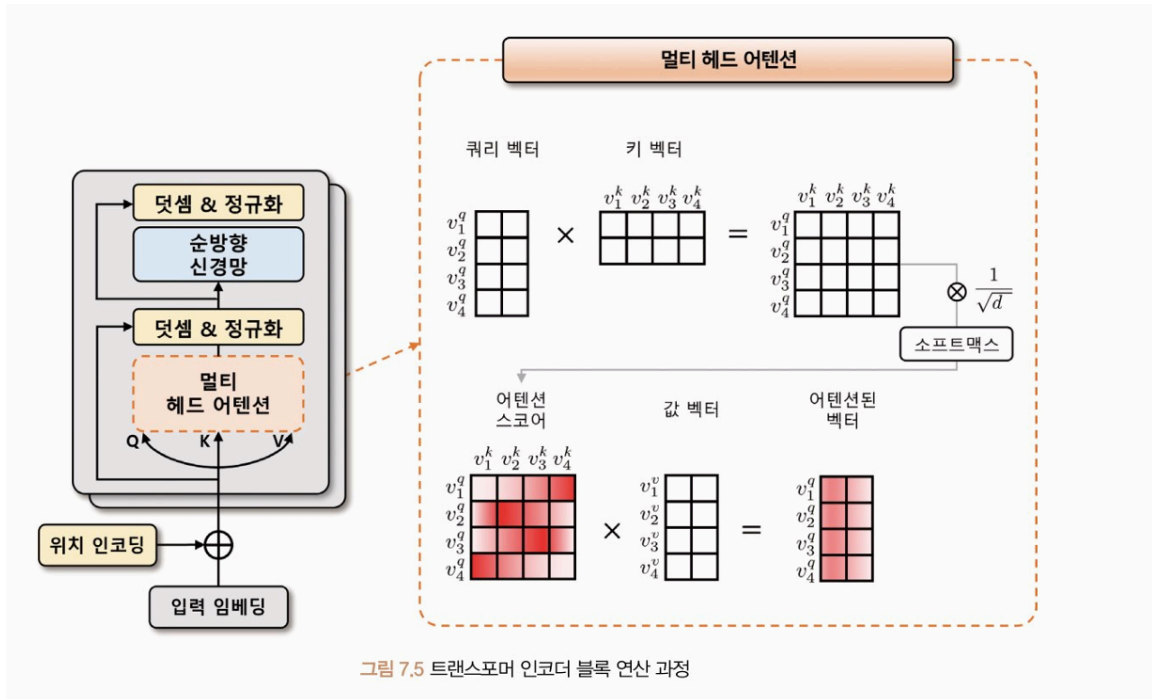


- N 개의 문장이 최대 S개의 토큰 길이를 가질 때 $[N, S, N]$ 크기의 원-핫 벡터 텐서는 $[N, S, d]$ 크기의 **임베딩 텐서**로 변환

트랜스포머 인코더

트랜스포머 인코더는 입력 시퀀스를 받아 여러 개의 계층으로 구성된 인코더 계층을 거쳐 연산을 수행. 인코더 계층에서 위치 정보를 반영하기 위해 위치 임베딩 벡터를 입력 벡터에 더해 줌. 산출된 출력을 디코더 계층으로 전달.

- 트랜스포머 인코더 블록의 연산 과정



위치 인코딩이 적용된 소스 데이터의 입력 임베딩을 입력받음

멀티 헤드 어텐션 단계에서 입력 텐서 차원 $[N, S, d]$ 일 때,

⇒ 선형 변환을 통해 3개의 임베딩 벡터를 생성

- **쿼리 (Q) 벡터** : 현재 시점에서 참조하고자 하는 정보의 위치를 나타냄. 이 벡터를 기준으로 다른 시점의 정보 참조
- **키 (K) 벡터** : 쿼리 벡터와 비교되는 대상으로, 쿼리 벡터를 제외한 입력 시퀀스에서 탐색되는 벡터. 인코더의 각 시점에서 생성
- **값 (V) 벡터** : 쿼리 벡터와 키 벡터로 생성된 어텐션 스코어를 얼마나 반영할지 설정하는 가중치 역할
 - 쿼리/키 벡터의 연관성은 내적 연산으로 $[N, S, S]$ 어텐션 스코어 맵을 사용



셀프 어텐션 과정

수식 7.2 어텐션 스코어 계산식

$$score(v^q, v^k) = \text{softmax}\left(\frac{(v^q)^T \cdot v^k}{\sqrt{d}}\right)$$

- 쿼리, 키 벡터를 내적해 어텐션 스코어를 구하고
- $d(1/2)$ (벡터 차원의 제곱근) 만큼 나눠 보정 → 벡터 차원이 커질 때 스코어값이 같이 커지는 문제 완화
- 보정된 어텐션 스코어를 소프트맥스 함수를 활용, 확률적으로 재표현

⇒ 값 벡터와 내적하여 셀프 어텐션된 벡터 생성, 여러 번 반복하여 셀프 어텐션된 어텐션 스코어 맵 생성

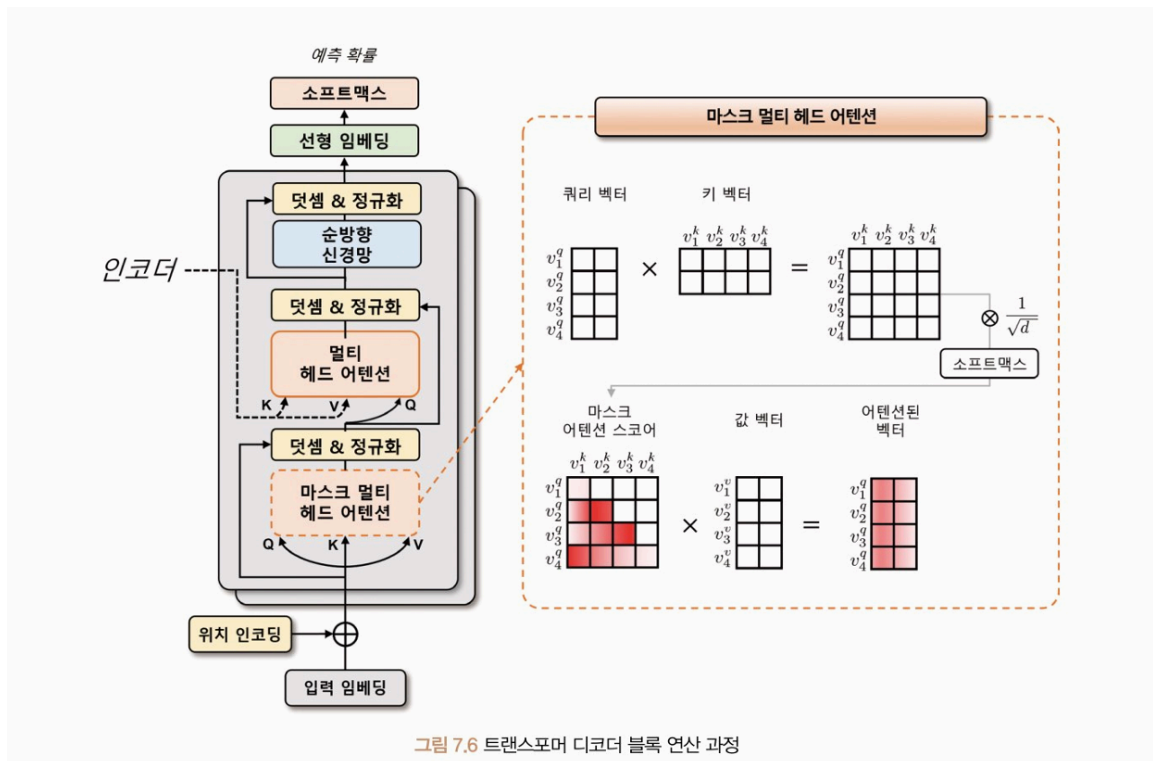
- 멀티 헤드 (Multi-Head) : 셀프 어텐션을 여러 번 수행해 여러 개의 헤드 생성. 여러 개의 헤드가 독립적으로 어텐션을 수행하고 그 결과를 합산
 - 입력받는 $[N, S, d]$ 텐서에 k 개의 셀프 어텐션 벡터를 생성하고자 한다면 헤드에 대한 차원 축을 생성해 $[N, k, S, d/k]$ 텐서 형태 구성
 - k 개의 셀프 어텐션 벡터는 임베딩 차원 축으로 병합(Concatenation) 되어 $[N, S, d]$ 형태로 출력
- 덧셈 & 정규화 : 멀티 헤드 어텐션을 통과하기 전의 입력값과 이후의 입력값을 더해 학습 시 발생하는 기울기 소실 문제 완화.
- 이 값에 임베딩 차원 축으로 정규화하는 계층 정규화 적용
- 순방향 신경망
 1. 선형 임베딩 & ReLU 로 이루어진 인공 신경망
 2. 1차원 합성곱

→ 덧셈 & 정규화 과정 수행

- 여러 개의 트랜스포머 인코더 블록으로 구성, 이전 블록에서 출력된 벡터는 다음 블록의 입력으로 전달되어 인코더 블록을 통과하면서 점차 입력 시퀀스의 정보가 추상화됨.
- 마지막 인코더 블록에서 출력된 벡터는 디코더에서 사용

트랜스포머 디코더

- 입력 : 위치 인코딩이 적용된 타겟 데이터의 입력 임베딩
→ 디코더가 입력 시퀀스의 순서 정보를 학습할 수 있게 함
- 인코더의 멀티 헤드 어텐션 모듈은 **인과성(Casuality)** 를 반영한 마스크 멀티 헤드 어텐션 모듈로 대체
- **마스크 멀티 헤드 어텐션 모듈** : 어텐션 스코어 맵을 계산할 때 첫 번째 쿼리 벡터가 첫 번째 키 벡터만을 바라볼 수 있게 마스크를 씌우고, 두 번째 쿼리 벡터는 첫, 두 번째 키 벡터를 바라보게 마스크를 씌움
→ 셀프 어텐션에서 현재 위치 이전의 단어들만 참조할 수 있어 인과성 보장
 - 마스크 영역에 수치적으로 굉장히 작은 값인 $-\infty$ 마스크를 더해 해당 영역의 어텐션 스코어값이 0이 가까워짐
→ 소프트맥스 계산 시 해당 영역의 어텐션 가중치가 0 이 되므로 마스크 영역 이전에 있는 입력 토큰들에 대한 정보를 참조하지 않게 됨.
- 트랜스포머 디코더 블록의 연산 과정



- 디코더의 멀티 헤드 어텐션 : 타겟 데이터(의 위치 정보를 포함한 입력 임베딩과 위치 임베딩을 더한 벡터)가 쿼리 벡터로 사용되며, 인코더의 소스 데이터가 키와 값 벡터로 사용됨.

⇒ 이전 시점에서의 정보를 현재 시점에서 활용할 수 있게 됨. 또한 마지막 디코더 블록의 출력 텐서에 선형 변환 및 소프트맥스 함수를 적용, 각 타겟 시퀀스 위치마다 예측 확률을 계산할 수 있게 함.

- 입력 토큰을 순차적으로 나타내는데, 이를 위해 PAD 토큰을 사용한다.
- 'GPT 는 트랜스포머 모델로 이뤄져 있다' 라는 문장을 추론할 때 디코더의 입력과 출력

표 7.2 인과성을 고려한 디코더 입력과 출력 예시

추론 순서	디코더 입력	디코더 출력
1	[BOS], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]
2	[BOS], 'ChatGPT', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]
3	[BOS], 'ChatGPT', '는', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', [PAD], [PAD], [PAD], [PAD], [PAD]
4	[BOS], 'ChatGPT', '는', '트랜스포머', [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', [PAD], [PAD], [PAD], [PAD]
5	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', [PAD], [PAD], [PAD]
6	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [PAD], [PAD]
7	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [EOS], [PAD]

실습

- Multi30k 데이터셋 사용 : 영어-독일어 병렬 말뭉치(Parallel corpus) 로 약 30000 개의 데이터를 제공.
 - `torchdata` 라이브러리 - 대규모 데이터셋을 다루기 쉽게 데이터를 불러오고 변환 및 배치하는 간단하고 유연한 API 제공
 - `torchtext` 라이브러리 - 파이토치 텍스트 처리 라이브러리.
 - `portalocker` 라이브러리 - 파이썬에서 파일 락을 관리하기 위한 라이브러리. 파일 락을 사용해 여러 프로세스 간에 동시에 파일을 수정하거나 읽는 것 방지
- 데이터셋 다운로드 및 전처리

```

from torchtext.datasets import Multi30k
from torchtext.data.utils import get_tokenizer
from torchtext.vocab import build_vocab_from_iterator

def generate_tokens(text_iter, language):
    language_index = {SRC_LANGUAGE: 0, TGT_LANGUAGE: 1}

    for text in text_iter:

```

```

yield token_transform[language](text[language_index[language]])

SRC_LANGUAGE = "de"
TGT_LANGUAGE = "en"

UNK_IDX, PAD_IDX, BOS_IDX, EOS_IDX = 0, 1, 2, 3
special_symbols = ["<unk>", "<pad>", "<bos>", "<eos>"]

token_transform = {
    SRC_LANGUAGE: get_tokenizer("spacy", language="de_core_news_sm",
    TGT_LANGUAGE: get_tokenizer("spacy", language="en_core_web_sm")
} #토크나이저, 어휘 사전 생성

print("Token Transform:")
print(token_transform)

vocab_transform = {} #토크를 인덱스로 변환시키는 함수 저장
for language in [SRC_LANGUAGE, TGT_LANGUAGE]: #독일어, 영어) 튜플 형
    train_iter = Multi30k(split="train", language_pair=(SRC_LANGUAGE, TGT
    vocab_transform[language] = build_vocab_from_iterator(
        generate_tokens(train_iter, language),
        min_freq=1, #최소 빈도: 토큰화된 단어들의 최소 빈도수 지정
        specials=special_symbols,
        special_first=True,
    )

for language in [SRC_LANGUAGE, TGT_LANGUAGE]:
    vocab_transform[language].set_default_index(UNK_IDX) #set_default_index

print("Vocab Transform:")
print(vocab_transform)

```

- 독일어/영어 말뭉치 `en/de_core_news_sm` 에 대해 각각 토크나이저와 어휘 사전 생성
- `get_tokenizer` 함수 : 사용자가 지정한 토크나이저를 가져오는 함수. spaCy 라이브러리
로 사전 학습된 함수 가져옴.
→
`token_transform` 변수에 저장

- `build_vocab_from_iterator` 함수 , `generate_tokens` 함수 : 언어별 어휘 사전 생성
- `specials` , `special_first` 변수 : 트랜스포머에 활용하는 특수 토큰을 지정, `special_first` 가 참인 경우 특수 토큰을 단어 집합의 맨 앞에 추가

- 트랜스포머 모델 구성

```
import math
import torch
from torch import nn

class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len, dropout=0.1):
        super().__init__()
        self.dropout = nn.Dropout(p=dropout)

        position = torch.arange(max_len).unsqueeze(1)
        div_term = torch.exp(
            torch.arange(0, d_model, 2) * (-math.log(10000.0) / d_model)
        )
        pe = torch.zeros(max_len, 1, d_model)
        pe[:, 0, 0::2] = torch.sin(position * div_term)
        pe[:, 0, 1::2] = torch.cos(position * div_term)
        self.register_buffer("pe", pe)

    def forward(self, x):
        x = x + self.pe[:x.size(0)]
        return self.dropout(x)

class TokenEmbedding(nn.Module):
    def __init__(self, vocab_size, emb_size):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, emb_size)
        self.emb_size = emb_size

    def forward(self, tokens):
        return self.embedding(tokens.long()) * math.sqrt(self.emb_size)
```

```

class Seq2SeqTransformer(nn.Module):
    def __init__(
        self,
        num_encoder_layers,
        num_decoder_layers,
        emb_size,
        max_len,
        nhead,
        src_vocab_size,
        tgt_vocab_size,
        dim_feedforward,
        dropout=0.1,
    ):
        super().__init__()
        self.src_tok_emb = TokenEmbedding(src_vocab_size, emb_size)
        self.tgt_tok_emb = TokenEmbedding(tgt_vocab_size, emb_size)
        self.positional_encoding = PositionalEncoding(
            d_model=emb_size, max_len=max_len, dropout=dropout
        )

        self.transformer = nn.Transformer( #트랜스포머 블록 -Transformer 클래스 쪽
            d_model=emb_size,
            nhead=nhead,
            num_encoder_layers=num_encoder_layers,
            num_decoder_layers=num_decoder_layers,
            #인코더와 디코더를 encoder_layers 변수의 값으로 구성
            dim_feedforward=dim_feedforward,
            dropout=dropout,
        )
        self.generator = nn.Linear(emb_size, tgt_vocab_size)
        #마지막 트랜스포머 디코더 블록에서 산출되는 벡터를 선형 변환, 어휘 사전에 대한

    def forward(
        self,
        src,
        trg,
        src_mask,
        tgt_mask,

```



```

src_padding_mask,
tgt_padding_mask,
memory_key_padding_mask,
):
    src_emb = self.positional_encoding(self.src_tok_emb(src))
    tgt_emb = self.positional_encoding(self.tgt_tok_emb(trg))
    outs = self.transformer(
        src=src_emb,
        tgt=tgt_emb,
        src_mask=src_mask,
        tgt_mask=tgt_mask,
        memory_mask=None,
        src_key_padding_mask=src_padding_mask,
        tgt_key_padding_mask=tgt_padding_mask,
        memory_key_padding_mask=memory_key_padding_mask,
    )
    return self.generator(outs)

def encode(self, src, src_mask):
    return self.transformer.encoder(
        self.positional_encoding(self.src_tok_emb(src)), src_mask
    )

def decode(self, tgt, memory, tgt_mask):
    return self.transformer.decoder(
        self.positional_encoding(self.tgt_tok_emb(tgt)), memory, tgt_mask
    )

```

`Seq2SeqTransformer` 클래스 : `TokenEmbedding` 클래스로 소스 데이터와 입력 데이터를 입력 임베딩으로 변환하여 `src_tok_emb` 와 `tgt_tok_emb` 생성

→ 소스와 타겟 데이터의 어휘 사전 크기를 입력받아 트랜스포머 임베딩 크기로 변환. 이 입력 임베딩에 `PositionalEncoding` 을 적용해 트랜스포머 블록에 입력

트랜스포머 클래스

- 임베딩 차원 (`d_model`) : 트랜스포머 모델의 입력과 출력 차원의 크기 정의
- 헤드 (`nhead`) : 멀티 헤드 어텐션의 개수 정의
- 인코더 계층 개수 (`num_encoder_layers`)
- 순방향 신경망 크기 (`dim_feedforward`)
- 드롭아웃 (`dropout`) : 인코더와 디코더 계층에 적용되는 드롭아웃 비율.

```
transformer = torch.nn.Transformer(  
    d_model=512,  
    nhead=8,  
    num_encoder_layers=6,  
    num_decoder_layers=6,  
    dim_feedforward=2048,  
    dropout=0.1,  
    activation=torch.nn.functional.relu,  
    layer_norm_eps=1e-05,  
)
```

- 계층 정규화 입실론 (`layer_norm_eps`) : 계층 정규화를 수행할 때 분모에 더해지는 입실론 값
- 활성화 함수 (`activation`)

트랜스포머 순방향 메서드

```
output = transformer.forward(  
    src,  
    tgt,  
    src_mask=None,  
    tgt_mask=None,  
    memory_mask=None,  
    src_key_padding_mask=None,  
    tgt_key_padding_mask=None,  
    memory_key_padding_mask=None,  
)
```

- 소스 (`src`) 와 타겟 (`tgt`) : 인코더와 디코더에 대한 시퀀스로 [소스(타겟) 시퀀스 길이, 배치 크기, 임베딩 차원] 형태의 데이터를 입력받음
- 소스 마스크 (`src_mask`) 와 타겟 마스크 (`tgt_mask`) : 소스와 타겟 시퀀스의 마스크 크기로 [소스(타겟) 시퀀스 길이, 시퀀스 길이] 형태의 데이터 입력받음.

⇒ 마스크의 값이 0이라면 해당 위치에서는 모든 입력 단어가 동일한 가중치를 갖고 어텐션이 수행, 1이라면 모든 입력 단어의 가중치가 0으로 설정돼 어텐션 연산이 수행 X, **-inf** 라면 해당 위치에서는 어텐션 연산 결과에 0으로 가중치가 부여돼 마스크된 위치의 정보를 모델이 무시하도록 할 수 있음. **+inf** 라면 모든 입력 단어에 대해 무한대의 가중치가 부여돼 어텐션 결과가 해당 위치에 대한 정보만으로 구성

- 트랜스포머 모델 구조

```
from torch import optim
```

```
BATCH_SIZE = 128
```

```
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
```

```

model = Seq2SeqTransformer(
    num_encoder_layers=3,
    num_decoder_layers=3,
    emb_size=512,
    max_len=512,
    nhead=8,
    src_vocab_size=len(vocab_transform[SRC_LANGUAGE]),
    tgt_vocab_size=len(vocab_transform[TGT_LANGUAGE]),
    dim_feedforward=512,
).to(DEVICE)

criterion = nn.CrossEntropyLoss(ignore_index=PAD_IDX).to(DEVICE)
#무시되는 색인 (ignore_index) 값을 패딩 토큰 (PAD_IDX) 를 할당해 모델이 학습하는 것
#패딩 토큰은 모델 학습에 사용되지 않음

optimizer = optim.Adam(model.parameters())

for main_name, main_module in model.named_children():
    print(main_name)
    for sub_name, sub_module in main_module.named_children():
        print("| ", sub_name)
        for ssub_name, ssub_module in sub_module.named_children():
            print("| | ", ssub_name)
            for sssub_name, sssub_module in ssub_module.named_children():
                print("| | | ", sssub_name)

```

Seq2SeqTransformer 클래스 구성 :

- 입력 임베딩 (src_tok_emb, tgt_tok_emb)
- 위치 인코딩 (positional_encoding)
- 트랜스포머 블록 (transformer)
 - 인코더 / 디코더 : 3개의 계층으로 구성
- 로짓 생성 (generator)

- 배치 데이터 생성

```

from torch.utils.data import DataLoader
from torch.nn.utils.rnn import pad_sequence

def sequential_transforms(*transforms):
    def func(txt_input):
        for transform in transforms:
            txt_input = transform(txt_input)
        return txt_input
    return func

def input_transform(token_ids):
    return torch.cat(
        (torch.tensor([BOS_IDX]), torch.tensor(token_ids), torch.tensor([EOS_IDX]
        )

def collator(batch):
    src_batch, tgt_batch = [], []
    for src_sample, tgt_sample in batch:
        src_batch.append(text_transform[SRC_LANGUAGE](src_sample.rstrip("\n"))
        tgt_batch.append(text_transform[TGT_LANGUAGE](tgt_sample.rstrip("\n"))

    src_batch = pad_sequence(src_batch, padding_value=PAD_IDX)
    tgt_batch = pad_sequence(tgt_batch, padding_value=PAD_IDX)
    return src_batch, tgt_batch

text_transform = {}
for language in [SRC_LANGUAGE, TGT_LANGUAGE]:
    text_transform[language] = sequential_transforms(
        token_transform[language], vocab_transform[language], input_transform
    )

data_iter = Multi30k(split="valid", language_pair=(SRC_LANGUAGE, TGT_LANGUAGE))
dataloader = DataLoader(data_iter, batch_size=BATCH_SIZE, collate_fn=collator)
source_tensor, target_tensor = next(iter(dataloader))

print("(source, target):")

```

```
print(next(iter(data_iter)))

print("source_batch:", source_tensor.shape)
print(source_tensor)

print("target_batch:", target_tensor.shape)
print(target_tensor)
```

Sequential_transforms 함수 : 여러 개의 전처리 함수를 인자로 받아 차례로 적용하는 함수를 반환

- ⇒ 전처리 1 : token_transform 적용해 문장 토큰화
- ⇒ 전처리 2 : vocab_transform 적용해 토큰 인덱스화
- ⇒ 전처리 3 : input_transform 함수 적용해 인덱스화된 토큰에 문장의 시작 BOS_IDX(2) 와 끝 EOS_IDX(3) 을 알리는 특수 토큰 할당

data_iter 함수 : (독일, 영어) 형태로 구성된 Multi30k 텍스트 데이터셋을 불러옴

- 데이터로더에 적용
- by (collator 집합 함수) : 배치 단위로 데이터 처리. 문자열의 끝에 있는 개행 문자를 제거하고 **text_transform** 변수에 저장된 **sequential_transforms** 함수 적용

pad_sequence 함수 - 소스와 타겟 시퀀스 패딩. (동일한 길이를 가지도록 시퀀스의 뒤쪽에 PAD_IDX(1) 로 채워진 패딩 토큰을 추가

데이터로더 dataloader : (패딩이 적용된 소스, 패딩이 적용된 타겟) 튜플 반환

- 어텐션 마스크 생성

```
def generate_square_subsequent_mask(s):
    mask = (torch.triu(torch.ones((s, s), device=DEVICE)) == 1).transpose(0, 1)
    mask = (
        mask.float()
        .masked_fill(mask == 0, float("-inf"))
        .masked_fill(mask == 1, float(0.0))
    )
    return mask

def create_mask(src, tgt):
    src_seq_len = src.shape[0]
```

```

tgt_seq_len = tgt.shape[0]

tgt_mask = generate_square_subsequent_mask(tgt_seq_len)
src_mask = torch.zeros((src_seq_len, src_seq_len), device=DEVICE).type(tc

src_padding_mask = (src == PAD_IDX).transpose(0, 1)
tgt_padding_mask = (tgt == PAD_IDX).transpose(0, 1)
return src_mask, tgt_mask, src_padding_mask, tgt_padding_mask

target_input = target_tensor[:-1, :]
target_out = target_tensor[1:, :]

source_mask, target_mask, source_padding_mask, target_padding_mask = cr
    source_tensor, target_input
)

print("source_mask:", source_mask.shape)
print(source_mask)

print("target_mask:", target_mask.shape)
print(target_mask)

print("source_padding_mask:", source_padding_mask.shape)
print(source_padding_mask)

print("target_padding_mask:", target_padding_mask.shape)
print(target_padding_mask)

```

패딩 마스크를 생성하는 함수 `create_mask`

- 시퀀스를 입력받아 길이를 계산하고 마스크 생성 함수로 타겟 시퀀스의 마스크를 생성

→ 소스 마스크, 패딩 마스크 생성

- 소스 마스크 → 소스 시퀀스 길이의 크기로 채워진 행렬 생성

트랜스포머 모델에서 사용되는 마스크를 생성하는 함수 `generate_square_subsequent_mask`

- 입력으로 정수 s 를 받아 $s \times s$ 크기의 마스크 생성
- `torch.ones` 함수 : 1로 채워진 행렬 생성
- `torch.triu` 함수 : 상삼각행렬 생성
- `transpose` 함수 : 행렬 전치

- 패딩 마스크 → 각 시퀀스에 대해 패딩 토큰의 위치를 찾고 transpose
패딩 마스크를 생성하기 전에 타겟 데이터의 입력값 (target_input) 과 출력값 (target_out) 토큰 순서를 한 칸 시프트하여 이전 토큰들로 다음 토큰을 예측하게 함.

→ 이 마스크 텐서에서 0인 값은 -inf (해당 타겟 시퀀스 셀프 어텐션에 참조 **✗**) 로, 1인 값은 0.0 (셀프 어텐션에 참조 **○**) 으로 채워 어텐션 연산 적용

- 모델 학습 및 평가

```
def run(model, optimizer, criterion, split):
    model.train() if split == "train" else model.eval()
    data_iter = Multi30k(split=split, language_pair=(SRC_LANGUAGE, TGT_LANGUAGE))
    dataloader = DataLoader(data_iter, batch_size=BATCH_SIZE, collate_fn=collate_fn)

    losses = 0
    for source_batch, target_batch in dataloader:
        source_batch = source_batch.to(DEVICE)
        target_batch = target_batch.to(DEVICE)

        target_input = target_batch[:-1, :]
        target_output = target_batch[1:, :]

        src_mask, tgt_mask, src_padding_mask, tgt_padding_mask = create_masks(
            source_batch, target_input
        )

        logits = model(
            src=source_batch,
            trg=target_input,
            src_mask=src_mask,
            tgt_mask=tgt_mask,
            src_padding_mask=src_padding_mask,
            tgt_padding_mask=tgt_padding_mask,
            memory_key_padding_mask=src_padding_mask,
```

```

    )

    optimizer.zero_grad()
    loss = criterion(logits.reshape(-1, logits.shape[-1]), target_output.reshape(-1, target_output.shape[-1]))
    if split == "train":
        loss.backward()
        optimizer.step()
    losses += loss.item()

return losses / len(list(dataloader))

for epoch in range(5):
    train_loss = run(model, optimizer, criterion, "train")
    val_loss = run(model, optimizer, criterion, "valid")
    print(f"Epoch: {epoch+1}, Train loss: {train_loss:.3f}, Val loss: {val_loss:.3f}")

```

- 트랜스포머 모델 번역 결과

```

def greedy_decode(model, source_tensor, source_mask, max_len, start_symbol):
    source_tensor = source_tensor.to(DEVICE)
    source_mask = source_mask.to(DEVICE)

    memory = model.encode(source_tensor, source_mask)
    ys = torch.ones(1, 1).fill_(start_symbol).type(torch.long).to(DEVICE)
    for i in range(max_len - 1):
        memory = memory.to(DEVICE)
        target_mask = generate_square_subsequent_mask(ys.size(0))
        target_mask = target_mask.type(torch.bool).to(DEVICE)

        out = model.decode(memory, ys, target_mask)
        out = out.transpose(0, 1)
        prob = model.generator(out[:, -1])
        _, next_word = torch.max(prob, dim=1)
        next_word = next_word.item()

        ys = torch.cat(
            [ys, torch.ones(1, 1).type_as(source_tensor.data).fill_(next_word)], dim=0
        )
    
```



```

    )

    if next_word == EOS_IDX:
        break
    return ys

def translate(model, source_sentence):
    model.eval()
    source_tensor = text_transform[SRC_LANGUAGE](source_sentence).view(-1, 1)
    num_tokens = source_tensor.shape[0]
    src_mask = (torch.zeros(num_tokens, num_tokens)).type(torch.bool)

    tgt_tokens = greedy_decode(
        model, source_tensor, src_mask, max_len=num_tokens + 5, start_symbol=1
    ).flatten()

    output = vocab_transform[TGT_LANGUAGE].lookup_tokens(list(tgt_tokens.cpu().numpy()))
    return " ".join(output)

output_oov = translate(model, "Eine Gruppe von Menschen steht vor einem Igloo")
output = translate(model, "Eine Gruppe von Menschen steht vor einem Gebäude")
print(output_oov)
print(output)

```

Greedy Decoding 방식

- 가장 높은 확률을 가진 단어를 매 시점마다 선택해 번역 결과를 생성하는 방식.
- 현재 시점의 확률만 고려하므로 **전역적인 최적 해를 보장하진 않음**.

사용되는 입력 및 마스크

- 인코더 입력: `source_tensor`, `source_mask`
- 디코더 입력: 이전에 예측한 토큰 시퀀스 `ys`, `target_mask`
- 인코더 출력 결과는 `memory` 텐서로 저장됨.

예측 과정

- `model.encode()` 로 인코더 블록 벡터(memory)를 얻고
- `model.decode()` 로 각 시점마다 다음 토큰의 확률 분포를 계산

- softmax 결과에서 가장 높은 확률의 토큰을 선택함
- 이 과정을 `max_len` 만큼 반복하거나 `EOS` 토큰이 나올 때까지 반복

토큰 처리

- `lookup_tokens()` 함수로 토큰 인덱스를 실제 텍스트로 변환
- BOS(<bos>), EOS(<eos>) 토큰은 슬라이싱으로 제거

GPT(Generative Pre-trained Transformer)

2018년 OpenAI 에서 발표한 트랜스포머 기반 언어 모델로 가장 널리 알려진 언어 모델. ELMo (Embeddings from Language Model) 과 같이 대규모 말뭉치로 사전 학습된 모델로 다양한 다운스트림 작업에서 우수한 성능을 보임.

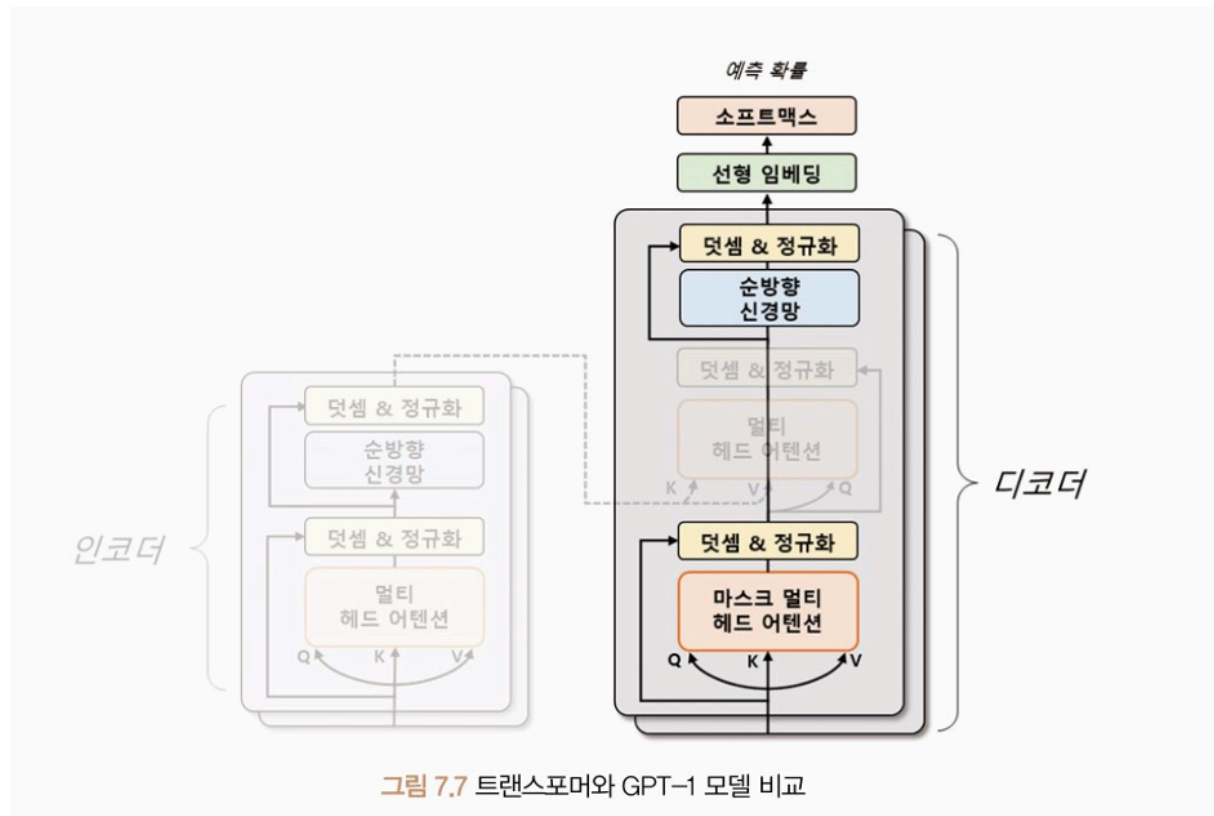
- GPT 모델은 트랜스포머의 디코더를 여러 층 쌓아 만든 언어 모델임
 - 디코더는 입력 문장과 출력 문장 간의 관계를 이해하고 문장을 생성하는데 초점
→ 디코더를 쌓아 모델을 구성하는 것이 자연어 생성과 같은 언어 모델링 작업에서 더 높은 성능을 발휘.
- GPT 모델 시리즈
 - GPT-1 : 1억 2천만 개의 매개변수가 존재
 - GPT-2 : 15억 개의 매개변수 존재
 - GPT-3 : 1750억 개의 매개변수를 갖는 초거대 모델, 많은 자원을 필요로 함
 - GPT-3.5 : 많은 양의 매개변수를 줄이기 위해 RLHF (Reinforcement Learning from Human Feedback) 방법을 도입.
 - RLHF = 인간의 피드백을 이용하는 강화 학습 모델로, 인간이 모델이 생성한 결과물을 평가하고 모델이 좋은 결과물을 생성하면 보상을 주어서 모델을 학습 시킴
 - 약 60억 개의 가중치로도 뛰어난 성능
 - Chat GPT 서비스의 기반
 - GPT-4 : 이미지 데이터를 함께 학습하는 모델. 텍스트 데이터에 국한되지 않고 이미지 데이터도 처리할 수 있게 개

GPT-1

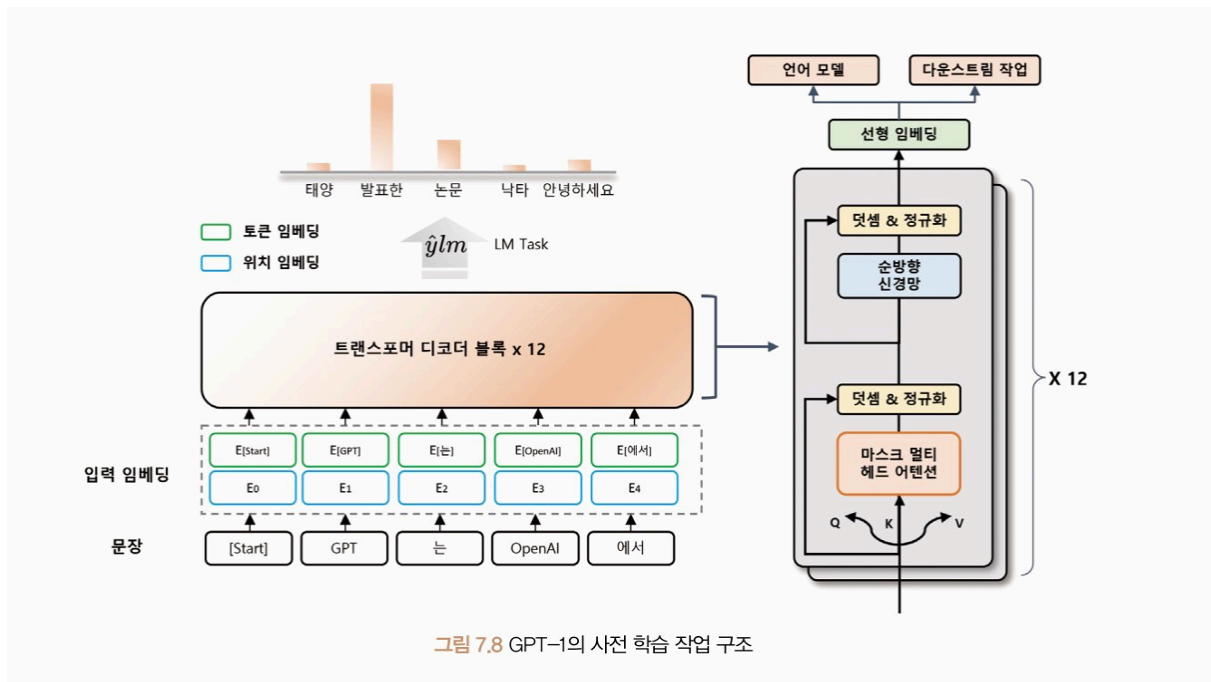
트랜스포머 구조를 바탕으로 한 단방향 (Unidirectional) 언어 모델

텍스트를 생성할 때 현재 위치에서 이전 단어들에 대한 정보만을 참고해 다음 단어를 예측

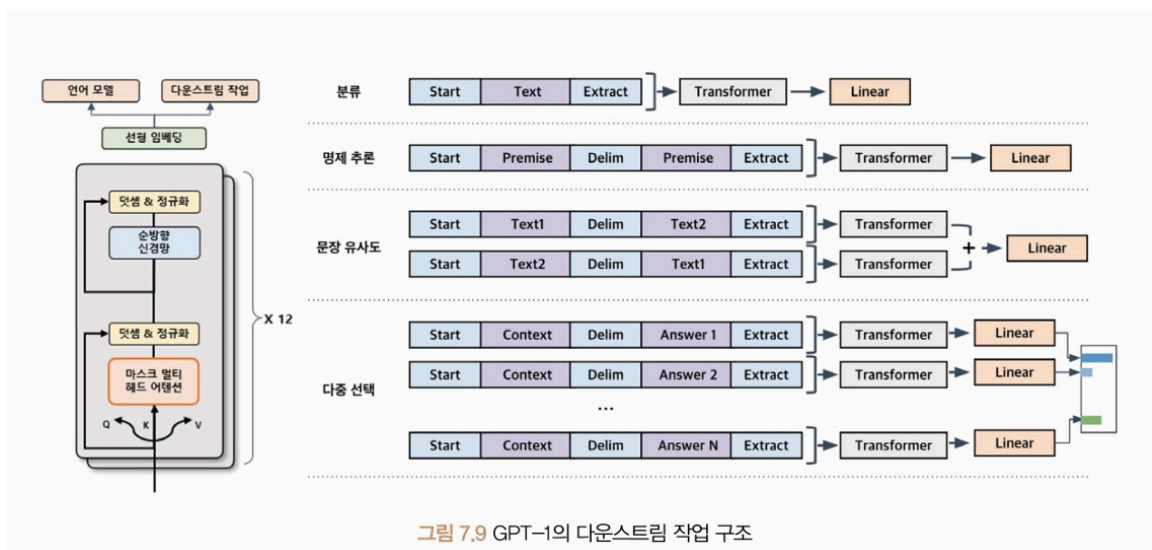
- 트랜스포머 구조 이용
- 디코더 부분만 사용하며, 인코더-디코더 간 어텐션 연산을 제외



- 디코더의 두 번째 다중헤드 어텐션 사용 X
- 12개의 헤드를 가진 트랜스포머의 디코더를 12층 “쌓아” 모델을 구성, 약 1억 2천만 개의 매개변수로 학습
- 4.5 GB 의 BookCorpus 데이터세트를 사용해 언어 모델 사전 학습
- 비지도 학습 → 컴퓨팅 자원만 충분하다면 제약 없이 학습할 수 있다.

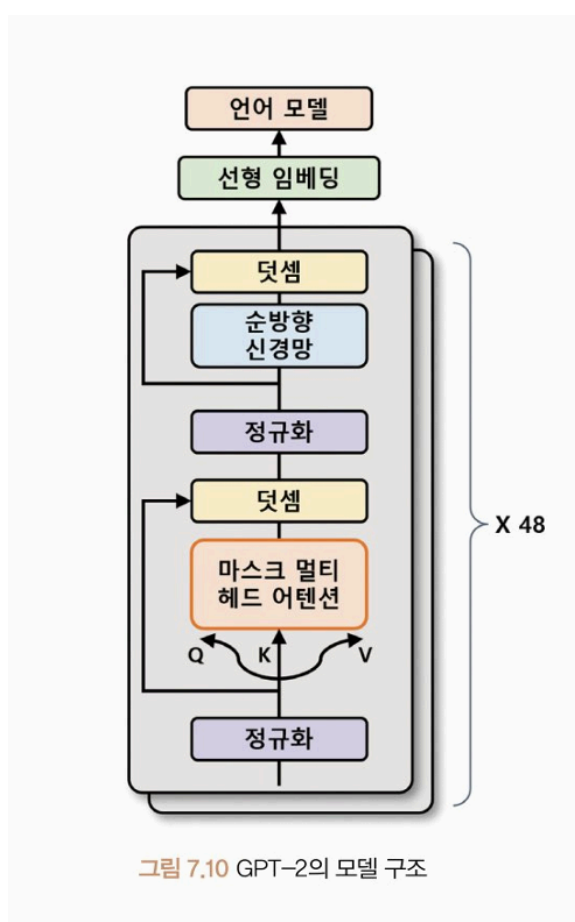


- 입력 문장을 일정한 길이의 블록으로 나누어 처리
 - 각 블록에서는 블록 내의 첫 번째 토큰부터 순서대로 다음 토큰을 예측하게 함
- ⇒ 모델이 장기적인 문맥을 이해할 수 있도록 해줌
- 입력 임베딩 계산 - 토큰 임베딩 (각 토큰을 고정된 차원의 벡터를 매핑) + 위치 임베딩 (입력 문장에서 각 토큰의 위치 정보를 나타내는 벡터를 매핑)
 - 미세 조정을 위한 다운스트림 - 각 작업에 따라 입출력 구조를 바꾸지 않고 언어 모델을 보조 학습 (Auxiliary Learning) 해 사용
 - 보조 학습 (Auxiliary Learning) : 모델이 주어진 주요 학습 작업 외에도 작은 부가적인 학습 작업을 동시에 수행하는 것



- 원하는 목표에 맞게 모델을 세밀하게 조정하는 작업이므로 추가 손실 함수가 필요
- 두 개의 손실 함수 (일반 학습시 사용되는 손실 함수 + 추가 손실 함수) 를 사용하면 보조 학습을 통해 언어 모델을 개선하는 동시에 다운스트림 작업의 목표를 달성하기 위해 필요한 정보 학습 가능
- 특수 토큰 사용 - <start>, <delim>, <extract>

GPT-2



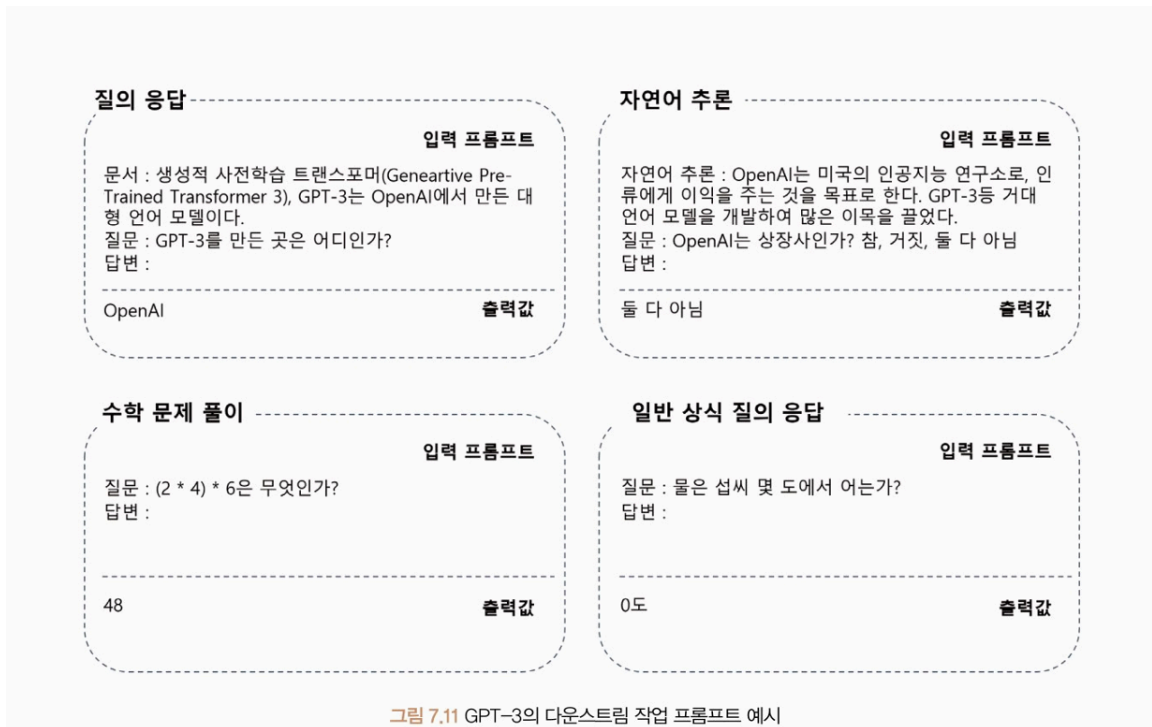
- 1024 길이까지 입력 문장을 받아들일 수 있음. (GPT-1 은 512)
- 48개의 디코더 계층 사용 → 더 복잡한 패턴 학습 가능
- 15억 개의 매개변수
- GPT -1 보다 훨씬 많은 데이터 - 미국의 웹사이트에서 스크래핑한 약 8백만 개의 문서를 사용해 40GB 의 데이터로 학습 수행
→ 더 자연스러운 문장 생성 가능
- **제로-샷 학습** : 별도의 미세 조정 없이 다운스트림 작업에서도 사용할 수 있게 사전 학습 과정에서 특정한 형식의 데이터 입력
ex) 'GPT는 놀라워요 = GPT is amazing' 형식 문장 학습 ⇒ '한국어 문장 =' 를 입력하면 번역된 문장이 출력하도록 작동

GPT-3

- GPT-2 보다 116배 많은 매개변수
- 매우 규모가 큰 모델로, 96개의 헤드를 가진 멀티 헤드 어텐션을 사용하며, 트랜스포머 디코더 층 역시 96개.

→ 연산량을 줄이기 위해 희소 어텐션 (Sparse Attention) 과 일반 어텐션 섞어 사용

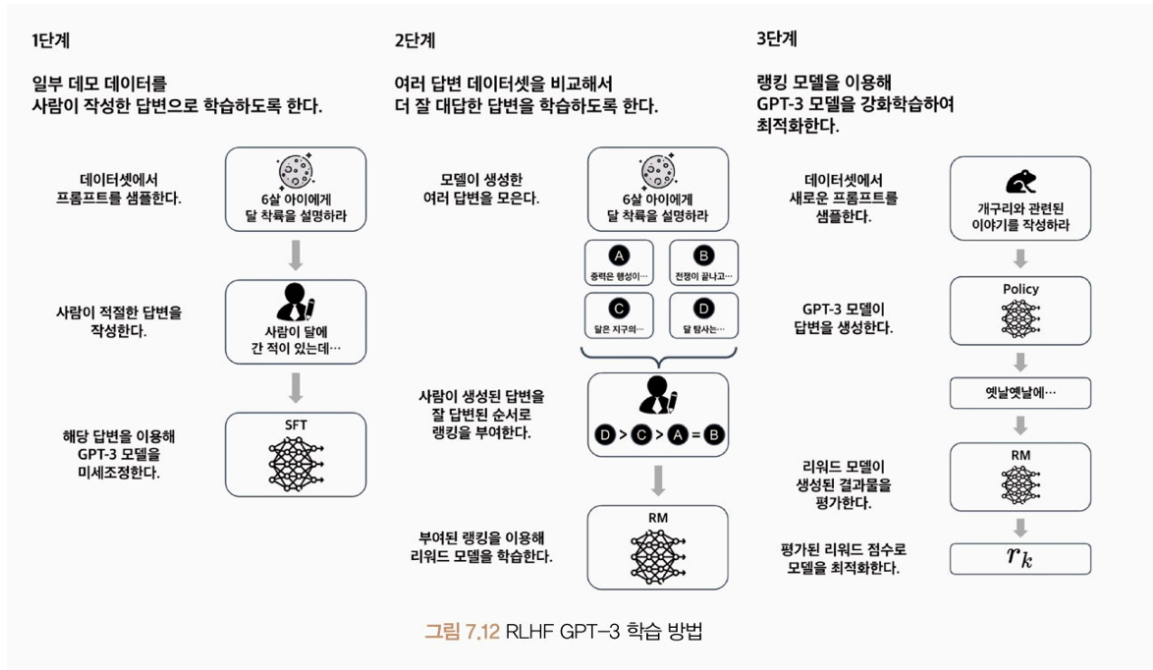
- 토큰 임베딩 크기 1600 → 12888, 2048개의 토큰을 입력할 수 있음
- 45TB 에 달하는 방대한 데이터세트 사용
- **프롬프트 (Prompt)** 형식으로 작업 수행



- 새로운 도메인에서의 작업에서 높은 일반화 능력, 기존 자연어 처리 분야에서는 이전의 기술적 한계를 극복하는 성과
- 하지만 큰 모델이기 때문에 매우 느리고 비용이 많이 듦.
- **인간적인 이해력과 상식적인 추론 능력 부족.** 모델이 생성한 텍스트가 항상 사실일 필요는 없으며, 편견이나 혐오 표현 등이 포함될 수 있다. → 검토, 검증 과정 필요

GPT 3.5 = Instruct GPT

- GPT 3의 구조에서 RLHF 를 도입해 모델의 매개변수 수를 줄이고 자연스러움을 높임
- RLHF



→ 강화 학습을 통해 모델 학습 : 지도 미세 조정 (Supervised Fine Tuning, SFT)

- 하지만 전체 프롬프트에 사람이 답변을 다는 것은 현실적으로 어려움 → GPT 모델이 얼마나 잘 답변했는지 평가하는 보상 (Reward) 모델 학습
- 주어진 프롬프트 (State) 에 대해 모델 (Policy) 가 답변을 생성 (Action) 하고 이를 이용해 사람의 피드백을 기반으로 한 학습된 보상 모델을 평가 (Reward)
- 언어 모델이 일부 입력에 대해 현실적이지 않은 결과를 생성하는 **환각 (Hallucination)** 현상 존재

GPT-4

- 이미지 데이터도 인식 가능한 멀티모달 (Multimodal) 모델
- 입력으로 최대 32768개 토큰 받을 수 있음
- 모델의 매개변수 수나 학습 데이터 및 방법은 공개되지 않음

모델 실습

허깅 페이스 트랜스포머 라이브러리의 GPT-2 모델로 문장 생성 실습

- 모델 구조

```

from transformers import GPT2LMHeadModel

model = GPT2LMHeadModel.from_pretrained("gpt2")

for main_name, main_module in model.named_modules():
    print(main_name)
    for sub_name, sub_module in main_module.named_modules():
        print(" |——", sub_name)
        for ssub_name, ssub_module in sub_module.named_modules():
            print(" | |——", ssub_name)
            for sssub_name, sssub_module in ssub_module.named_modules():
                print(" | | |——", sssub_name)

```

- GPT2LMHeadModel 클래스의 from_pretrained 메서드로 사전 학습된 GPT-2 모델 불러오기
- 단어 토큰 임베딩 (wte), 단어 위치 임베딩 (wpe), 드롭아웃 (drop), 트랜스포머 디코더 계층(h), 선형 임베딩 및 언어 모델 (lm_head) 로 구성

```

transformer
├── wte
├── wpe
├── drop
├── h
│   ├── 0
│   │   ├── ln_1
│   │   ├── attn
│   │   ├── ln_2
│   │   └── mlp
│   ├── 1
│   │   ├── ln_1
│   │   ├── attn
│   │   ├── ln_2
│   │   └── mlp
│   ├── 2
│   │   ├── ln_1
│   │   ├── attn
│   │   ├── ln_2
│   │   └── mlp
│   ├── 3
│   │   ├── ln_1
│   │   ├── attn
│   │   ├── ln_2
│   │   └── mlp
│   └── ...
│       ├── ln_2
│       └── mlp
└── ln_f
    └── lm_head

```

Output is truncated. View as a [scrollable](#)

- GPT-2 를 이용한 문장 생성

#7-10 gpt-2 를 이용한 문장 생성

```

from transformers import pipeline

```

```

generator = pipeline(task="text-generation", model="gpt2")
outputs = generator(

```



```

text_inputs="Machine learning is",
max_length=20,
num_return_sequences=3,
pad_token_id=generator.tokenizer.eos_token_id
)

print(outputs)

```

pipeline 클래스 : 입력된 작업(text-generation)과 모델(gpt2)로 적합한 파이프라인 구축.

- 설정한 작업을 수행하는 파이프라인 클래스의 인스턴스 반환

ex) "text-generation" 작업 → TextGenerationPipeline 클래스의 인스턴스 생성

입력 텍스트 (text_inputs) : 생성하려는 문장의 입력 문맥 전달

최대 길이 (max_length) :: 생성될 문장의 최대 토큰 수 제한

반환 시퀀스 개수 (num_return_sequences): 생성할 텍스트 시퀀스의 수 의미

패딩 토큰 ID (pad_token_id) : 모델의 자유 생성 (Open-end generation) 여부 설정

- CoLA 데이터셋 불러오기

```

import torch
from torchtext.datasets import CoLA
from transformers import AutoTokenizer
from torch.utils.data import DataLoader

def collator(batch, tokenizer, device):
    source, labels, texts = zip(*batch)
    tokenized = tokenizer(
        texts,
        padding="longest",
        truncation=True,
        return_tensors="pt"
    )
    input_ids = tokenized["input_ids"].to(device)
    attention_mask = tokenized["attention_mask"].to(device)
    labels = torch.tensor(labels, dtype=torch.long).to(device)

```

```

    return input_ids, attention_mask, labels

train_data = list(CoLA(split="train"))
valid_data = list(CoLA(split="dev"))
test_data = list(CoLA(split="test"))

tokenizer = AutoTokenizer.from_pretrained("gpt2")
#토큰 아이디와 어텐션 마스크 반환
tokenizer.pad_token = tokenizer.eos_token #eos 토큰으로 대체

epochs = 3
batch_size = 16
device = "cuda" if torch.cuda.is_available() else "cpu"

train_dataloader = DataLoader(
    train_data,
    batch_size=batch_size,
    collate_fn=lambda x: collator(x, tokenizer, device),
    shuffle=True,
)

valid_dataloader = DataLoader(
    valid_data,
    batch_size=batch_size,
    collate_fn=lambda x: collator(x, tokenizer, device),
)

test_dataloader = DataLoader(
    test_data,
    batch_size=batch_size,
    collate_fn=lambda x: collator(x, tokenizer, device),
)

print("Train Dataset Length :", len(train_data))
print("Valid Dataset Length :", len(valid_data))
print("Test Dataset Length :", len(test_data))

```

- CoLA 데이터세트는 train, dev, test 로 구성

- GPT-2 는 사전 학습 시 패딩 기법을 사용하지 않기 때문에 문장의 끝을 의미하는 eos 토큰으로 패딩 토큰을 대체

- GPT-2 모델 설정

```
from torch import optim
from transformers import GPT2ForSequenceClassification

model = GPT2ForSequenceClassification.from_pretrained(
    pretrained_model_name_or_path="gpt2",
    num_labels=2 #CoLA 데이터세트에 올바른 문장/아닌 문장으로 레이블이 두 가지이.
).to(device)

model.config.pad_token_id = model.config.eos_token_id

optimizer = optim.Adam(model.parameters(), lr=5e-5)
```

- GPT-2 모델 학습 및 검증

```
import numpy as np
from torch import nn

def calc_accuracy(preds, labels):
    pred_flat = np.argmax(preds, axis=1).flatten()
    labels_flat = labels.flatten()
    return np.sum(pred_flat == labels_flat) / len(labels_flat)

def train(model, optimizer, dataloader):
    model.train()
    train_loss = 0.0

    for input_ids, attention_mask, labels in dataloader:
        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask,
            labels=labels
```

```

    )

    loss = outputs.loss
    train_loss += loss.item()

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    train_loss = train_loss / len(dataloader)
    return train_loss

def evaluation(model, dataloader):
    with torch.no_grad():
        model.eval()
        criterion = nn.CrossEntropyLoss()
        val_loss, val_accuracy = 0.0, 0.0

    for input_ids, attention_mask, labels in dataloader:
        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask,
            labels=labels
        )
        logits = outputs.logits

        loss = criterion(logits, labels)
        logits = logits.detach().cpu().numpy()
        label_ids = labels.to("cpu").numpy()
        accuracy = calc_accuracy(logits, label_ids)

        val_loss += loss
        val_accuracy += accuracy

    val_loss = val_loss / len(dataloader)
    val_accuracy = val_accuracy / len(dataloader)

    return val_loss, val_accuracy

```

```

# 학습 루프
best_loss = 10000
for epoch in range(epochs):
    train_loss = train(model, optimizer, train_dataloader)
    val_loss, val_accuracy = evaluation(model, valid_dataloader)
    print(f"Epoch {epoch + 1}: Train Loss: {train_loss:.4f} Val Loss: {val_loss:.4f} Val Accuracy: {val_accuracy:.4f}")

    if val_loss < best_loss:
        best_loss = val_loss
        torch.save(model.state_dict(), "GPT.pt")
        print("Saved the model weights")

```

```

Epoch 1: Train Loss: 0.4998 Val Loss: 0.5163 Val Accuracy: 0.7543
Saved the model weights
Epoch 2: Train Loss: 0.3611 Val Loss: 0.5219 Val Accuracy: 0.7562
Epoch 3: Train Loss: 0.2552 Val Loss: 0.5331 Val Accuracy: 0.7652

```

- 모델 평가

```

from transformers import GPT2ForSequenceClassification
import torch

# 모델 로드 및 설정
model = GPT2ForSequenceClassification.from_pretrained(
    pretrained_model_name_or_path="gpt2",
    num_labels=2
).to(device)

# 패딩 토큰 ID 설정
model.config.pad_token_id = model.config.eos_token_id

# 저장된 가중치 로드
model.load_state_dict(torch.load("../models/GPT2ForSequenceClassification.pt"))

# 평가

```

```
test_loss, test_accuracy = evaluation(model, test_dataloader)

print(f"Test Loss : {test_loss:.4f}")
print(f"Test Accuracy : {test_accuracy:.4f}")
```